Trie Data Structure Explained

Absolutely. Let's build **Trie from zero**, step by step, so you understand not only the code but **why Trie exists, how it works internally, and how to recognize Trie questions in DSA**.

1. What is a Trie?

A **Trie** (pronounced **"try"**) is a tree-based data structure used mainly to store and search **strings/prefixes efficiently**.

The most important idea is:

> **Characters are stored along the edges/path of a tree, and words sharing a prefix share the same path.**

For example, suppose we insert:

```
cat
car
can
dog
```

The Trie looks conceptually like:

```
               root
              /    \
             c      d
             |      |
             a      o
          /  |  \   |
         t   r   n  g
```

Notice something important:

```
cat
car
can
```

all start with:

```
ca
```

So they share the same nodes:

```
root
  |
  c
  |
  a
 /|\
t r n
```

This **prefix sharing** is the main reason we use Trie.

---

2. Why not just use an array or unordered_map?

Suppose we have:

```
apple
application
apply
banana
bat
ball
```

If we want to find whether **`"application"`** exists, we could use:

```
C++
unordered_set<string>
```

and lookup is approximately:

```
O(1)
```

So why Trie?

Because Trie is extremely good when the problem is about **prefixes**.

For example:

> Find all words starting with **`"app"`**.

With a normal hash set, you potentially need to examine many strings.

With Trie:

```
root
 |
 a
 |
 p
 |
 p
 /|\
l l ...
```

We directly walk:

```
a → p → p
```

and now we are at the node representing prefix **`"app"`**.

This makes Trie especially useful for:

- prefix searching
- autocomplete
- dictionary problems
- word searching
- spelling systems
- word replacement
- XOR-related problems using binary Trie
- finding maximum/minimum XOR
- prefix-based string problems

---

3. Mental model of a Trie

Think of Trie as a **dictionary represented as a tree**.

Suppose we have:

```
apple
app
ape
bat
```

We create:

```
                root
               /    \
              a      b
              |      |
              p      a
              |      |
              p      t
             /  \
            l    e
            |
            e
```

But there is one more important thing.

How do we know whether:

```
app
```

is a complete word or just a prefix of:

```
apple
```

We need a special variable:

```
C++
isEnd
```

So:

```
apple
app
```

would look like:

```
root
 |
 a
 |
 p
 |
 p  ← isEnd = true
 |
 l
 |
 e  ← isEnd = true
```

The **app** node tells us:

> A complete word ends here.

The **apple** node tells us:

> Another complete word ends here.

This **isEnd** concept is **extremely important**.

---

4. Structure of a Trie Node

For lowercase English letters:

```
C++
struct Node {
    Node* child[26];
    bool isEnd;
};
```

Each node contains:

1. Children

```
C++
Node* child[26];
```

Why 26?

Because:

```
a b c d e ... z
```

There are 26 lowercase letters.

We map:

```
a → 0
b → 1
c → 2
...
z → 25
```

using:

```
C++
int index = ch - 'a';
```

For example:

```
ch = 'c'

'c' - 'a' = 2
```

So:

```
C++
child[2]
```

represents character **'c'**.

---

5. Why pointers?

Suppose we have:

```
root
 |
 c
```

The root needs to point to the node representing `c`.

So:

```
C++
Node* child[26];
```

means:

```
            root
      / / / / | \ \ \
     a b c d ...     z
         |
        Node
```

Most of these pointers are initially:

```
C++
nullptr
```

Only the characters that actually exist get nodes.

---

6. Creating a Node

We can write:

```
C++
struct Node {
    Node* child[26];
    bool isEnd;

    Node() {
        isEnd = false;

        for(int i = 0; i < 26; i++) {
            child[i] = nullptr;
        }
    }
};
```

Initially:

```
isEnd = false

child[0] = nullptr
child[1] = nullptr
...
child[25] = nullptr
```

7. Trie Class

Now:

```
C++
class Trie {
    Node* root;
};
```

The root represents:

```
""
```

the empty string.

It does **not** represent any character.

So:

```
          root
         /    \
        a      b
        |      |
        p      a
```

The root is simply the starting point.

---

8. Insert Operation

Now let's understand the most important operation.

Suppose we insert:

```
"cat"
```

Start at:

```
root
```

Step 1

Character:

```
c
```

Convert:

```
C++
index = 'c' - 'a'
      = 2
```

Check:

```
C++
root->child[2]
```

If it doesn't exist:

```
C++
root->child[2] = new Node();
```

Move there:

```
C++
root = root->child[2];
```

Now:

```
root
 |
 c
```

Step 2

Next character:

```
a
```

Create the **a** node.

```
root
 |
 c
 |
 a
```

Step 3

Next character:

```
t
```

Create:

```
root
 |
 c
 |
 a
 |
 t
```

Now we reached the end of the word.

So:

```
C++
isEnd = true;
```

Final:

```
root
 |
 c
 |
 a
 |
 t  ← isEnd = true
```

That's insertion.

---

9. Insert another word

Now insert:

```
car
```

Start:

```
root → c
```

c already exists.

So **don't create a new node**.

Move:

```
root
 |
 c
```

Next:

```
a
```

Already exists.

Move:

```
root
 |
 c
 |
 a
```

Next:

```
r
```

Doesn't exist.

Create it.

Now:

```
       root
         |
         c
         |
         a
        / \
       t   r
```

So Trie automatically shares:

```
ca
```

between:

```
cat
car
```

10. Insert `can`

Insert:

```
can
```

c exists.

a exists.

n doesn't exist.

Create **n**.

Now:

```
          root
            |
            c
            |
            a
          / | \
         t  r  n
```

All three words share:

```
ca
```

This is the fundamental Trie optimization:

> **Common prefixes are stored only once.**

---

11. Complete Insert Code

```
C++
class Trie {
    Node* root;

public:

    Trie() {
        root = new Node();
    }

    void insert(string word) {

        Node* curr = root;

        for(char ch : word) {

            int index = ch - 'a';

            if(curr->child[index] == nullptr) {
                curr->child[index] = new Node();
            }

            curr = curr->child[index];
        }

        curr->isEnd = true;
    }
};
```

12. Search Operation

Now suppose:

```
Trie contains:

cat
car
can
```

We want:

```
search("car")
```

Start at root.

```
root
 |
 c
 |
 a
 /|\
t r n
```

Follow:

```
c → a → r
```

If every character exists, we reach the `r` node.

Then check:

```
C++
curr->isEnd
```

If:

```
C++
true
```

then:

```
"car" exists
```

13. Important case: Prefix vs Word

Suppose Trie contains:

```
apple
```

Now search:

```
app
```

Path exists:

```
a → p → p
```

But:

```
C++
isEnd == false
```

because no word ended at **`app`**.

Therefore:

```
"app" does NOT exist
```

Even though:

```
"app" is a prefix of "apple"
```

This is why **`isEnd`** is necessary.

---

14. Search Code

```
C++
bool search(string word) {

    Node* curr = root;

    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return false;
        }

        curr = curr->child[index];
    }

    return curr->isEnd;
}
```

15. `startsWith()` Operation

This is where Trie becomes really powerful.

Suppose:

```
Trie:
```

```
apple
application
apply
banana
```

Question:

```
startsWith("app")
```

We don't care whether **"app"** itself is a word.

We only care whether its path exists.

So:

```
a → p → p
```

exists.

Therefore:

```
true
```

Code

```C++
bool startsWith(string prefix) {
    Node* curr = root;
    for(char ch : prefix) {
        int index = ch - 'a';
        if(curr->child[index] == nullptr) {
            return false;
        }
        curr = curr->child[index];
    }
    return true;
}
```

Notice the difference:

Search

```C++
return curr->isEnd;
```

Prefix

```C++
return true;
```

after successfully walking the entire prefix.

---

16. Three Fundamental Trie Operations

For the basic Trie, remember:

Operation	Meaning
`insert(word)`	Store a word
`search(word)`	Check complete word
`startsWith(prefix)`	Check whether prefix exists

These three are the standard Trie operations.

---

17. Time Complexity

Suppose word length is:

```
L
```

Insert

We process every character once:

```
O(L)
```

Search

Again:

```
O(L)
```

Prefix Search

Again:

```
O(L)
```

So Trie operations depend on the **length of the word**, not directly on the number of stored words.

This is one of the major advantages of Trie.

---

18. Complete Implementation

Here is the standard LeetCode-style implementation:

```
C++
class Trie {

    struct Node {
        Node* child[26];
        bool isEnd;

        Node() {
            isEnd = false;

            for(int i = 0; i < 26; i++) {
                child[i] = nullptr;
            }
        }
    };

    Node* root;

public:

    Trie() {
        root = new Node();
    }

    void insert(string word) {

        Node* curr = root;

        for(char ch : word) {

            int index = ch - 'a';

            if(curr->child[index] == nullptr) {
                curr->child[index] = new Node();
            }

            curr = curr->child[index];
```

19. Let's Dry Run the Code

Suppose:

```
C++
Trie trie;

trie.insert("apple");
trie.insert("app");
trie.insert("ape");
```

After insertion:

```
                root
                 |
                 a
                 |
                 p
               /   \
              p     e*
              |     |
              l     ...
              |
              e*
```

* means:

```
isEnd = true
```

So:

```
ape  → complete word
app  → complete word
apple → complete word
```

20. Search `"app"`

We follow:

```
root
 ↓
 a
 ↓
 p
 ↓
 p*
```

At **p**:

```
C++
isEnd = true
```

Therefore:

```
true
```

21. Search `"ap"`

We follow:

```
a → p
```

But:

```
C++
isEnd = false
```

So:

```
false
```

Even though `"ap"` is a prefix.

22. Search `"application"`

We follow:

```
a
 ↓
p
 ↓
p
 ↓
l
 ↓
...
```

At some point:

```
C++
curr->child[index] == nullptr
```

So immediately:

```
C++
return false;
```

23. How Trie Questions Appear in DSA

This is probably the **most important part for competitive programming**.

When you see a problem involving:

```
prefix
dictionary
autocomplete
words
characters
search
starts with
lexicographical
XOR
maximum XOR
minimum XOR
word replacement
```

you should think:

> **Can Trie help?**

24. Common Trie Problem Pattern #1 — Dictionary

Example:

```
insert("apple")
insert("app")

search("apple") → true
search("ap")    → false
startsWith("ap") → true
```

This is the basic Trie problem.

LeetCode:

```
208. Implement Trie (Prefix Tree)
```

This is the problem you should solve first.

25. Pattern #2 — Autocomplete

Imagine Google search.

You type:

```
app
```

Trie might contain:

```
apple
application
app
appointment
apply
```

The Trie takes you to:

```
a → p → p
```

Then from this node we can explore its descendants:

```
         app
       /  |  \
     le   ly   lication
```

and generate suggestions.

That's why Trie is naturally suited for:

> "Give me all words starting with this prefix."

---

26. Pattern #3 — Word Replacement

Suppose:

```
dictionary:

cat
bat
rat
```

Sentence:

```
the cattle was running
```

We can use Trie to find whether some prefix of:

```
cattle
```

matches a dictionary root:

```
cat
```

Since:

```
cat
```

is a prefix of:

```
cattle
```

we can replace:

```
cattle → cat
```

This is another common Trie pattern.

---

27. Pattern #4 — Maximum XOR

This is a **very important advanced Trie application**.

Suppose:

```
numbers:

5
10
25
```

and we want:

```
maximum XOR
```

We can represent every number in binary:

```
5  = 00101
```

```
10 = 01010
25 = 11001
```

Instead of storing characters:

```
a,b,c...
```

we store:

```
0 and 1
```

So Trie becomes:

```
            root
           /    \
          0      1
         / \    / \
        0   1  0   1
```

This is called a:

> **Binary Trie**

It's extremely common in competitive programming.

---

28. Trie Is Not Only for Strings

This is another important concept.

You may initially think:

> Trie = strings.

Not exactly.

Trie is really:

> **A tree where paths represent sequences.**

The sequence can be:

```
characters
bits
digits
words
states
```

For example:

```
String Trie:
a → p → p → l → e
```

Binary Trie:

```
1 → 0 → 1 → 0 → 1
```

So the underlying idea is:

```
Sequence → Path in Tree
```

29. Why Trie Can Be Better Than HashMap

Suppose:

```
100000 words
```

and each word has length:

```
20
```

A hash table is excellent for:

```
"Does this exact word exist?"
```

But suppose the question is:

```
How many words start with "pre"?
```

Trie naturally represents:

```
p → r → e
```

and we can maintain information at that node.

For example:

```
C++
prefixCount
```

Then:

```
prefixCount at "pre"
```

could directly tell us:

```
How many words start with "pre"
```

This leads to more advanced Trie implementations.

---

30. Advanced Trie Node

Instead of:

```
C++
struct Node {
    Node* child[26];
    bool isEnd;
};
```

we can have:

```
C++
struct Node {
    Node* child[26];
    bool isEnd;
    int prefixCount;
    int wordCount;
};
```

Now the Trie can answer more complicated questions.

For example:

```
How many words have prefix "app"?
```

or:

```
How many times was "apple" inserted?
```

31. Trie Deletion

Another operation is:

```
delete(word)
```

This is more complicated than insertion/search.

Suppose:

```
cat
car
```

Trie:

```
      c
      |
      a
     / \
    t   r
```

Delete:

```
cat
```

We don't necessarily delete `c` or `a`, because:

```
car
```

still needs them.

So we remove only unnecessary nodes.

After deleting **`cat`**:

```
    c
    |
    a
     \
      r
```

The important idea:

> **Only remove nodes that are no longer needed by any other word.**

For basic DSA learning, implement **`insert`**, **`search`**, and **`startsWith`** first. Learn deletion afterward.

---

32. The Most Important Mental Model

When solving a Trie problem, think like this:

```
                Trie
                 |
        ┌────────┴────────┐
        ↓                 ↓
   Character          Bit
      |                 |
String Trie       Binary Trie
      |                 |
  prefix             XOR
  search           problems
  words
  dictionary
  autocomplete
```

And internally:

```
          root
           |
      ┌────┼────┐
      a    b    c
      |         |
      p         a
      |         |
      p         t
```

Every path represents a sequence.

---

33. How to Recognize a Trie Problem

When reading a problem, ask these questions:

Question 1

> Are we dealing with many strings?

If yes → Trie may be useful.

Question 2

> Is the problem asking about prefixes?

If yes → **strong Trie signal.**

Question 3

> Do we repeatedly insert/search words?

If yes → Trie is likely useful.

Question 4

> Do we need autocomplete?

If yes → Trie.

Question 5

> Is it about XOR of numbers?

Think:

> **Binary Trie**

Question 6

> Do we need lexicographical traversal?

Trie can help because children can be explored from:

```
0 → 25
```

or:

```
'a' → 'z'
```

34. Trie vs HashMap vs Array

A useful comparison:

Feature	HashMap/Set	Trie
Exact word search	Excellent	Excellent
Prefix search	Poor/needs extra work	Excellent
Insert	O(L) average	O(L)
Search	O(L) average	O(L)
Autocomplete	Not natural	Natural
Memory	Usually lower	Can be high
Implementation	Easy	More complex
XOR problems	No	Binary Trie works

Here `L` = length of the word.

---

35. The Biggest Disadvantage of Trie

Trie can consume **a lot of memory**.

If every node contains:

```
C++
Node* child[26];
```

then every node has 26 pointers.

Suppose you have:

```
1 million nodes
```

that's a lot of memory.

That's why Trie implementations sometimes use:

```
C++
unordered_map<char, Node*>
```

instead of:

```
C++
Node* child[26]
```

For lowercase English letters, however, the fixed array is usually faster and simpler.

---

36. Your Learning Path for Trie

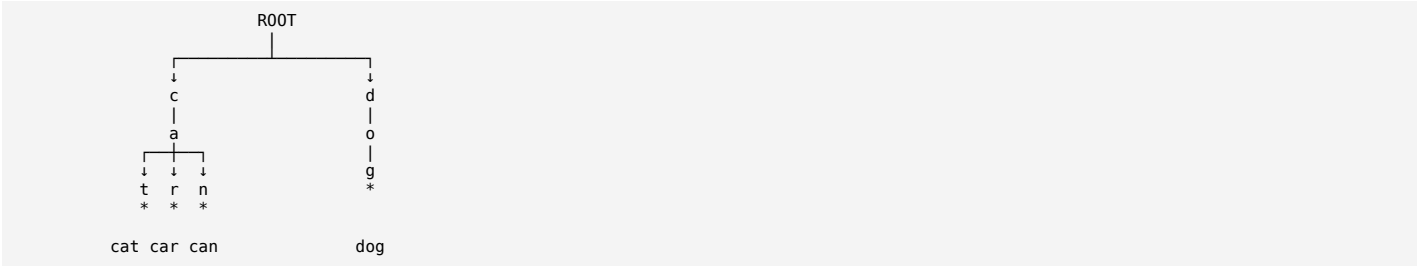
I recommend learning it in this exact order:

```
            TRIE
              |
              ↓
       1. Understand Node
```

```
              |
              ↓
       2. Understand root
              |
              ↓
       3. Insert
              |
              ↓
       4. Search
              |
              ↓
       5. startsWith
              |
              ↓
       6. Prefix counting
              |
              ↓
       7. Delete
              |
              ↓
       8. Autocomplete
              |
              ↓
       9. Binary Trie
              |
              ↓
      10. Maximum XOR
```

37. One Final Mental Picture

Keep this picture in your head:



Here:

```
root → c → a → t = "cat"
root → c → a → r = "car"
root → c → a → n = "can"
root → d → o → g = "dog"
```

The * means:

```
isEnd = true
```

And the key idea is:

```
                WORD
                  ↓
            sequence of chars
                  ↓
             path in Trie
                  ↓
      common prefixes share nodes
```

So if you remember only **one sentence**, remember:

> **A Trie stores strings as paths in a tree, where common prefixes share the same path, making prefix-based operations very efficient.**

If you're practicing DSA, the next natural step is to take **LeetCode 208 (Implement Trie)** and do a **full visual dry run of `insert()`, `search()`, and `startsWith()` line-by-line**, then move to **Trie questions like Word Search II, Replace Words, and Maximum XOR using Binary Trie**.

Yes. These are the **next level of Trie operations**. Once you understand **`isEnd`**, the key idea is to replace it with **counters**.

Instead of storing only:

```
C++
bool isEnd;
```

we store:

```
C++
int wordCount;
int prefixCount;
```

These two variables make all three operations easy:

```
countWordsEqualTo(word)
countWordsStartingWith(prefix)
erase(word)
```

Let's build this from the beginning.

---

1. Why do we need `wordCount` and `prefixCount`?

Suppose we insert:

```
apple
apple
app
application
app
```

Our Trie needs to know:

```
"apple" appears 2 times

"app" appears 2 times

How many words start with "app"?
= 5
```

A simple:

```
C++
bool isEnd;
```

cannot tell us this.

It can only say:

```
Does a word end here?
YES / NO
```

So we change:

```
C++
bool isEnd;
```

to:

```
C++
int wordCount;
int prefixCount;
```

2. Meaning of `wordCount`

`wordCount` means:

> **How many times exactly this word has been inserted.**

For example:

```
insert("apple")
insert("apple")
insert("apple")
```

At the **`e`** node:

```
wordCount = 3
```

So:

```
C++
countWordsEqualTo("apple")
```

returns:

```
3
```

3. Meaning of `prefixCount`

`prefixCount` means:

How many inserted words pass through this node.

This is extremely important.

Suppose we insert:

```
apple
app
application
ape
```

Look at the node representing:

```
app
```

The words passing through **app** are:

```
apple
app
application
```

Therefore:

```
prefixCount("app") = 3
```

ape does not pass through **app**.

So:

```
prefixCount("app") = 3
```

4. Visualize it

Let's insert:

```
apple
app
application
ape
```

Conceptually:

```
                      root
                        |
                        a
                        |
                        p
                        |
                        p
                       /  \
                      l    e
                      |    |
                      e    ?
```

Actually, for:

```
apple
app
application
```

the tree continues:

```
                        a
                        |
                        p
                        |
                        p
                       / \
                      l   ...
                      |
                      e
                      |
                      ...
```

At the **app** node:

```
wordCount = 1
prefixCount = 3
```

Why?

Because:

```
app           ← exact word
apple         ← starts with app
```

```
application   ← starts with app
```

5. The Node structure

So our Node becomes:

```
C++
struct Node {
    Node* child[26];

    int wordCount;
    int prefixCount;

    Node() {
        wordCount = 0;
        prefixCount = 0;

        for(int i = 0; i < 26; i++) {
            child[i] = nullptr;
        }
    }
};
```

Now each node stores three types of information:

```
                    Node
                     |
        ┌────────────┼────────────┐
        ↓            ↓            ↓
   children    wordCount    prefixCount
```

6. What happens during insertion?

This is the most important part.

Suppose:

```
C++
insert("apple");
```

Start:

```
root
```

Then process:

```
a → p → p → l → e
```

Every time we enter a node, increase:

```
C++
prefixCount++;
```

So:

```
root
 |
 a       prefixCount = 1
 |
 p       prefixCount = 1
 |
 p       prefixCount = 1
 |
 l       prefixCount = 1
 |
 e       prefixCount = 1
```

At the end:

```
C++
wordCount++;
```

So the final node has:

```
wordCount = 1
prefixCount = 1
```

7. Insert `apple` again

Now:

```
C++
insert("apple");
```

No new nodes are created.

We follow:

```
a → p → p → l → e
```

and increment **`prefixCount`** for every node.

Now:

```
a → prefixCount = 2
p → prefixCount = 2
p → prefixCount = 2
l → prefixCount = 2
e → prefixCount = 2
```

At **`e`**:

```
wordCount = 2
```

Therefore:

```
apple
```

exists **2 times**.

---

8. Insert `app`

Now:

```
C++
insert("app");
```

Follow:

```
a → p → p
```

Increment:

```
prefixCount
```

So:

```
a → 3
p → 3
p → 3
```

At the final **`p`**:

```
C++
wordCount++;
```

Therefore:

```
wordCount("app") = 1
prefixCount("app") = 3
```

Why **`prefixCount = 3`**?

Because these three words pass through **`app`**:

```
apple
apple
app
```

9. The complete insertion algorithm

```
C++
void insert(string word) {

    Node* curr = root;

    for(char ch : word) {
```

```
        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            curr->child[index] = new Node();
        }

        curr = curr->child[index];

        curr->prefixCount++;
    }

    curr->wordCount++;
}
```

Notice the order:

```
move to node
      ↓
prefixCount++
```

Then after the entire word:

```
wordCount++
```

10. Now `countWordsEqualTo()`

Question:

```
countWordsEqualTo("apple")
```

Meaning:

> How many times was exactly **`"apple"`** inserted?

We simply walk:

```
a → p → p → l → e
```

If at any point the node doesn't exist:

```
return 0
```

Otherwise, at the final node:

```
C++
return curr->wordCount;
```

Implementation:

```
C++
int countWordsEqualTo(string word) {

    Node* curr = root;

    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return 0;
        }

        curr = curr->child[index];
    }

    return curr->wordCount;
}
```

11. Example

Suppose we inserted:

```
apple
apple
app
```

Trie information:

```
             root
               |
               a
               |
               p
               |
```

```
              p
              |
              l
              |
              e
```

At final **e**:

```
wordCount = 2
```

So:

```
C++
countWordsEqualTo("apple")
```

returns:

```
2
```

12. Important difference

Suppose:

```
apple
apple
app
application
```

At node:

```
app
```

we have:

```
wordCount = 1
prefixCount = 4
```

Why?

`wordCount` asks:

> Exactly how many **`"app"`**?

Answer:

```
1
```

`prefixCount` asks:

> How many inserted words start with **`"app"`**?

Answer:

```
4
```

because:

```
apple
apple
app
application
```

all start with **`"app"`**.

This distinction is **the heart of these advanced Trie operations**.

---

13. `countWordsStartingWith()`

Now:

```
C++
countWordsStartingWith("app")
```

means:

> How many inserted words have **`"app"`** as their prefix?

We don't need to go beyond the **p** node.

We walk:

```
a → p → p
```

Then simply return:

```
C++
curr->prefixCount;
```

Implementation:

```
C++
int countWordsStartingWith(string prefix) {

    Node* curr = root;

    for(char ch : prefix) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return 0;
        }

        curr = curr->child[index];
    }

    return curr->prefixCount;
}
```

14. Why does this work?

Suppose:

```
insert("apple")
insert("apple")
insert("app")
insert("application")
insert("banana")
```

At:

```
app
```

the words passing through are:

```
apple
apple
app
application
```

Therefore:

```
prefixCount = 4
```

So:

```
C++
countWordsStartingWith("app")
```

returns:

```
4
```

No need to search the entire subtree.

That's the power of maintaining **`prefixCount`**.

---

15. Now the interesting one: `erase()`

Suppose:

```
Trie contains:

apple
apple
app
application
```

Now:

```
C++
erase("apple");
```

What should happen?

Before:

```
apple → 2 occurrences
```

After:

```
apple → 1 occurrence
```

We should **not necessarily delete the nodes**.

Instead, decrease the counts.

---

16. What happens when we erase?

We follow:

```
a → p → p → l → e
```

For every node:

```
C++
prefixCount--;
```

At the final node:

```
C++
wordCount--;
```

So:

```
Before erase:

a  prefixCount = 4
p  prefixCount = 4
p  prefixCount = 4
l  prefixCount = 3
e  prefixCount = 2
```

After erasing one **"apple"**:

```
a  prefixCount = 3
p  prefixCount = 3
p  prefixCount = 3
l  prefixCount = 2
e  prefixCount = 1
```

And:

```
e.wordCount = 1
```

17. But there's an important problem

What if we do:

```
C++
erase("hello");
```

when **"hello"** doesn't exist?

We must **not decrease anything**.

Otherwise our counts become incorrect.

So first we need to determine:

> Does this word exist?

There are two common approaches.

Approach 1

Walk through the Trie first.

If the path doesn't exist:

```
C++
return;
```

If it exists but:

```
C++
wordCount == 0
```

then the word isn't currently stored.

Otherwise erase.

---

18. Simple erase implementation

```
C++
void erase(string word) {

    Node* curr = root;

    // Step 1: Check whether word exists
    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return;
        }

        curr = curr->child[index];
    }

    // Word doesn't exist
    if(curr->wordCount == 0) {
        return;
    }

    // Step 2: Traverse again and decrease prefixCount
    curr = root;

    for(char ch : word) {

        int index = ch - 'a';

        curr = curr->child[index];

        curr->prefixCount--;
    }

    // Step 3: Remove one occurrence
    curr->wordCount--;
}
```

This works.

But we can make it more efficient.

---

19. Better `erase()` implementation

We don't need to traverse twice.

We can store the nodes we visit.

Or even simpler, because the specification guarantees we only erase one occurrence if it exists, we can first find the terminal node and then traverse again.

For beginner understanding, the previous implementation is actually easier to reason about.

Complexity is still:

```
O(L)
```

because:

```
O(L) + O(L) = O(2L) = O(L)
```

20. Full Advanced Trie

Now let's put everything together.

```
C++
```

```
}

void insert(string word) {

    Node* curr = root;

    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            curr->child[index] = new Node();
        }

        curr = curr->child[index];

        curr->prefixCount++;
    }

    curr->wordCount++;
}

int countWordsEqualTo(string word) {

    Node* curr = root;

    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return 0;
        }

        curr = curr->child[index];
    }

    return curr->wordCount;
}

int countWordsStartingWith(string prefix) {

    Node* curr = root;

    for(char ch : prefix) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return 0;
        }

        curr = curr->child[index];
    }

    return curr->prefixCount;
}

void erase(string word) {

    Node* curr = root;

    // Check if word exists
    for(char ch : word) {

        int index = ch - 'a';

        if(curr->child[index] == nullptr) {
            return;
        }

        curr = curr->child[index];
    }
```

```
        if(curr->wordCount == 0) {
            return;
        }

        // Decrease prefix counts
        curr = root;

        for(char ch : word) {

            int index = ch - 'a';

            curr = curr->child[index];

            curr->prefixCount--;
        }

        // Remove one occurrence
        curr->wordCount--;
    }
};
```

21. Let's do a complete dry run

This is where the concept should become very clear.

Start with an empty Trie.

Execute:

```
C++
insert("apple");
insert("apple");
insert("app");
insert("application");
```

After first `apple`

```
root
 |
 a [prefix=1]
 |
 p [prefix=1]
 |
 p [prefix=1]
 |
 l [prefix=1]
 |
 e [prefix=1, word=1]
```

After second `apple`

```
root
 |
 a [prefix=2]
 |
 p [prefix=2]
 |
 p [prefix=2]
 |
 l [prefix=2]
 |
 e [prefix=2, word=2]
```

Insert `app`

We stop at the second **p**.

```
root
 |
 a [prefix=3]
 |
 p [prefix=3]
 |
 p [prefix=3, word=1]
 |
 l [prefix=2]
 |
 e [prefix=2, word=2]
```

Notice:

```
app:
wordCount = 1
prefixCount = 3
```

because:

```
apple
apple
app
```

pass through **app**.

Insert `application`

The path:

```
app
```

already exists.

We continue:

```
l → i → c → a → t → i → o → n
```

At every node, increment:

```
prefixCount
```

Now at **app**:

```
prefixCount = 4
```

because:

```
apple
apple
app
application
```

all pass through it.

---

22. Queries now

Query 1

```
C++
countWordsEqualTo("apple")
```

Go:

```
a → p → p → l → e
```

At **e**:

```
wordCount = 2
```

Answer:

```
2
```

Query 2

```
C++
countWordsEqualTo("app")
```

Go:

```
a → p → p
```

At **p**:

```
wordCount = 1
```

Answer:

```
1
```

Query 3

```
C++
countWordsStartingWith("app")
```

Go:

```
a → p → p
```

At **p**:

```
prefixCount = 4
```

Answer:

```
4
```

Query 4

```
C++
countWordsStartingWith("apple")
```

At **e**:

```
prefixCount = 2
```

Answer:

```
2
```

because:

```
apple
apple
```

23. Now erase `apple`

Before:

```
apple → 2 occurrences
```

Call:

```
C++
erase("apple");
```

We decrement:

```
a prefixCount: 4 → 3
p prefixCount: 4 → 3
p prefixCount: 4 → 3
l prefixCount: 2 → 1
e prefixCount: 2 → 1
```

and:

```
apple.wordCount:
2 → 1
```

Now:

```
C++
countWordsEqualTo("apple")
```

returns:

```
1
```

But:

```
C++
countWordsStartingWith("app")
```

returns:

```
3
```

because now only:

```
apple
app
application
```

remain.

---

24. Why don't we physically delete nodes?

This is a very important question.

Suppose we have:

```
apple
app
```

Trie:

```
      a
      |
      p
      |
      p
      |
      l
      |
      e
```

If we erase:

```
apple
```

we cannot simply delete:

```
a
p
p
```

because **app** still needs them.

So we simply change counts:

```
apple.wordCount = 0
```

and reduce prefix counts.

The nodes can remain.

This is often called **logical deletion**.

---

25. But can we physically delete nodes?

Yes.

If after deleting a word a node has:

```
prefixCount == 0
```

then no remaining word uses that node.

For example:

```
Only:

apple
```

exists.

After:

```
C++
erase("apple");
```

we get:

```
e.prefixCount = 0
l.prefixCount = 0
```

```
p.prefixCount = 0
p.prefixCount = 0
a.prefixCount = 0
```

Those nodes are no longer needed.

You could physically delete them.

But for most competitive programming problems, **you don't need to physically delete nodes**.

Just maintain the counts.

---

26. The key difference between the two counters

This is the part I want you to remember most:

```
              Node
               |
       ┌───────┴────────┐
       ↓                ↓
  wordCount        prefixCount
       |                |
       ↓                ↓
Exact word         Prefix/subtree
```

`wordCount`

Answers:

> How many times does **exactly this word** occur?

`prefixCount`

Answers:

> How many words **pass through this prefix**?

---

27. Think of every node as representing a prefix

For example:

```
root
 |
 a        ← prefix = "a"
 |
 p        ← prefix = "ap"
 |
 p        ← prefix = "app"
 |
 l        ← prefix = "appl"
 |
 e        ← prefix = "apple"
```

At every node we can ask:

```
How many words end here?
        ↓
   wordCount

How many words pass here?
         ↓
   prefixCount
```

This mental model makes the entire implementation much easier.

---

28. Complexity

For a word of length `L`:

Insert

```
O(L)
```

`countWordsEqualTo`

```
O(L)
```

`countWordsStartingWith`

```
O(L)
```

erase

```
O(L)
```

So all four operations are:

```
        O(L)
```

where `L` is the length of the input string.

---

29. The advanced Trie template to memorize

For competitive programming, this is the structure you should remember:

```
C++
struct Node {
    Node* child[26];

    int wordCount;
    int prefixCount;

    Node() {
        wordCount = 0;
        prefixCount = 0;

        for(int i = 0; i < 26; i++)
            child[i] = nullptr;
    }
};
```

Then remember the four operations conceptually:

```
INSERT
    ↓
walk through characters
    ↓
prefixCount++
    ↓
at final node: wordCount++

SEARCH EXACT
    ↓
walk through characters
    ↓
return wordCount

SEARCH PREFIX
    ↓
walk through prefix
    ↓
return prefixCount

ERASE
    ↓
check word exists
    ↓
walk through characters
    ↓
prefixCount--
    ↓
final node: wordCount--
```

That is the **core advanced Trie pattern**.

Once this is comfortable, the next major jump is **Trie + DFS** for problems like **Word Search II**, and then **Binary Trie (0/1 Trie)** for maximum/minimum XOR. Those two patterns are where Trie starts becoming very powerful in competitive programming.